SQL Part 2 – Terminology Concepts Reference

Database Systems

Based on Lecture 4 by Tiantian Liu

Fall 2025

Contents

1 Quick Reference Guide **2**

2 SQL Command Order - Step-by-Step Guide **2**
- 2.1 Step-by-Step Checklist . . . 2
- 2.2 Quick Examples Following the Order . . . 4
- 2.3 Common Mistakes - What NOT to Do . . . 5
- 2.4 Quick Reference Card . . . 5

3 Null Values **6**
- 3.1 Testing for NULL . . . 6
- 3.2 NULL in Comparisons . . . 7
- 3.3 NULL with Boolean Operations . . . 7

4 Modification of the Database **9**
- 4.1 Deletion . . . 9
- 4.2 Insertion . . . 9
- 4.3 Updates . . . 10
- 4.4 CASE Statement in UPDATE . . . 10

5 Join Expressions **11**
- 5.1 Natural Join . . . 11
- 5.2 Join with USING Clause . . . 12
- 5.3 Join with ON Clause . . . 13
- 5.4 Outer Joins . . . 13

6 Nested Subqueries **14**
- 6.1 Subqueries in FROM, WHERE, SELECT . . . 15
- 6.2 Set Membership (IN and NOT IN) . . . 15
- 6.3 Set Comparison (SOME and ALL) . . . 16
- 6.4 Test for Empty Relations (EXISTS) . . . 17
- 6.5 Test for Duplicate Tuples (UNIQUE) . . . 18
- 6.6 WITH Clause . . . 19

7 Views, Transactions and Integrity Constraints **19**
- 7.1 Views . . . 19
- 7.2 Transactions . . . 20
- 7.3 Integrity Constraints . . . 20
- 7.4 Referential Integrity . . . 21

7.5 Handling Updates - Referential Integrity 21
7.6 Cascading Actions 22

8 Summary **24**

1 Quick Reference Guide

Common Patterns - At a Glance

NULL Handling:

```
WHERE attribute IS NULL      -- Check for NULL
WHERE attribute IS NOT NULL  -- Check for non-NULL
```

Database Modification:

```
DELETE FROM table WHERE condition;
INSERT INTO table VALUES (v1, v2, ...);
UPDATE table SET col = value WHERE condition;
```

Join Operations:

```
NATURAL JOIN                  -- All common columns
JOIN ... USING (col)         -- Specific columns
JOIN ... ON condition        -- General condition
LEFT/RIGHT/FULL OUTER JOIN   -- Preserve unmatched
```

Nested Subqueries:

```
WHERE attr IN (SELECT ...)
WHERE attr > ALL/SOME (SELECT ...)
WHERE EXISTS (SELECT ...)
WHERE UNIQUE (SELECT ...)
```

2 SQL Command Order - Step-by-Step Guide

Complete SQL Query Structure

Write SQL commands in this EXACT order:

```
SELECT [DISTINCT] columns
FROM table(s)
[WHERE condition]
[GROUP BY columns]
[HAVING condition]
[ORDER BY columns]
[LIMIT number]
```

Note: [ ] means optional. You must follow this order!

2.1 Step-by-Step Checklist

Writing a SQL Query - Follow These Steps

STEP 1: Start with SELECT

- Write `SELECT` first
- List the columns you want to see

- Use `DISTINCT` if you want unique rows only

```
SELECT name, salary
SELECT DISTINCT major
```

STEP 2: Add FROM

- Write `FROM` next
- Specify which table(s) to query
- Use JOIN here if combining tables

```
FROM instructor
FROM students JOIN enrollments ON students.id = enrollments.id
```

STEP 3: Add WHERE (optional)

- Write `WHERE` to filter rows
- Only includes rows that match the condition
- Cannot use aggregates like AVG, COUNT here

```
WHERE salary > 70000
WHERE major = 'CS' AND age > 20
```

STEP 4: Add GROUP BY (optional)

- Write `GROUP BY` to group rows
- Use when you have aggregate functions
- Groups rows with same values

```
GROUP BY dept_name
GROUP BY major, age
```

STEP 5: Add HAVING (optional)

- Write `HAVING` to filter groups
- Only after GROUP BY
- Can use aggregates like AVG, COUNT here

```
HAVING AVG(salary) > 50000
HAVING COUNT(*) > 5
```

STEP 6: Add ORDER BY (optional)

- Write `ORDER BY` to sort results
- Use ASC (ascending) or DESC (descending)
- This is always near the end

```
ORDER BY salary DESC
ORDER BY name ASC, age DESC
```

STEP 7: Add LIMIT (optional)

- Write `LIMIT` last
- Restricts number of rows returned
- Always the final clause

```
LIMIT 10
```

2.2 Quick Examples Following the Order

Example 1: Simple Query (Steps 1-2)

```
SELECT name, salary
FROM instructor;
```

Only uses SELECT and FROM.

Example 2: With Filter (Steps 1-3)

```
SELECT name, salary
FROM instructor
WHERE salary > 70000;
```

Uses SELECT, FROM, WHERE.

Example 3: With Grouping (Steps 1-4-5)

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 50000;
```

Uses SELECT, FROM, GROUP BY, HAVING.
Notice: No WHERE in this example (it's optional).

Example 4: Complete Query (All Steps)

```
SELECT dept_name, AVG(salary) AS avg_sal
FROM instructor
WHERE salary IS NOT NULL
GROUP BY dept_name
HAVING AVG(salary) > 50000
ORDER BY avg_sal DESC
LIMIT 5;
```

Uses all clauses in correct order.

2.3 Common Mistakes - What NOT to Do

Wrong Order

WRONG - WHERE after GROUP BY:

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
WHERE salary > 70000;  -- ERROR! Wrong order
```

CORRECT - WHERE before GROUP BY:

```
SELECT dept_name, AVG(salary)
FROM instructor
WHERE salary > 70000
GROUP BY dept_name;  -- Correct order!
```

WRONG - Aggregate in WHERE:

```
SELECT dept_name
FROM instructor
WHERE AVG(salary) > 50000;  -- ERROR! Use HAVING
```

CORRECT - Aggregate in HAVING:

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 50000;  -- Correct!
```

WRONG - ORDER BY before GROUP BY:

```
SELECT dept_name, COUNT(*)
FROM instructor
ORDER BY dept_name
GROUP BY dept_name;  -- ERROR! Wrong order
```

CORRECT - GROUP BY before ORDER BY:

```
SELECT dept_name, COUNT(*)
FROM instructor
GROUP BY dept_name
ORDER BY dept_name;  -- Correct order!
```

2.4 Quick Reference Card

Remember This Order

1. **SELECT** - What columns to show
2. **FROM** - Which table(s)
3. **WHERE** - Filter rows (before grouping)
4. **GROUP BY** - Group rows

5. **HAVING** - Filter groups (after grouping)
6. **ORDER BY** - Sort results
7. **LIMIT** - Limit number of results

Memory tip: Sally **F**rom **W**here **G**oes **H**ome **O**n **L**eave
Key rules:

- WHERE comes BEFORE GROUP BY
- HAVING comes AFTER GROUP BY
- ORDER BY is always near the end
- WHERE = filter rows, HAVING = filter groups
- Aggregates (AVG, COUNT, SUM) only in SELECT/HAVING, NOT in WHERE

3 Null Values

NULL Concept

NULL signifies an unknown value or that a value does not exist.
Key Properties:

- Any arithmetic expression involving NULL returns NULL
- Example: `5 + NULL` returns `NULL`
- Comparisons with NULL return UNKNOWN (not TRUE or FALSE)

3.1 Testing for NULL

NULL Predicates

Use special predicates to test for NULL values:

```
WHERE attribute IS NULL        -- Checks if NULL
WHERE attribute IS NOT NULL    -- Checks if not NULL
```

NEVER use: `= NULL` or `<> NULL` (these don't work!)

NULL Check Examples from Lecture

Find instructors whose salary is NULL:

```
SELECT name
FROM instructor
WHERE salary IS NULL;
```

Find instructors with a salary assigned:

```
SELECT name, salary
FROM instructor
```

```
WHERE salary IS NOT NULL;
```

3.2 NULL in Comparisons

Three-Valued Logic

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN

- Any comparison with NULL returns UNKNOWN
- `NULL = NULL` returns UNKNOWN (not TRUE!)
- `NULL <> NULL` returns UNKNOWN
- `5 < NULL` returns UNKNOWN

In WHERE clause: UNKNOWN is treated as FALSE (row excluded from result)

NULL Comparison Example from Lecture

```
SELECT name, salary
FROM instructor
WHERE salary > 70000;
```

If an instructor has `salary = NULL`:

- `NULL > 70000` evaluates to UNKNOWN
- UNKNOWN is treated as FALSE in WHERE clause
- This instructor is NOT included in the result

3.3 NULL with Boolean Operations

Boolean Operations with UNKNOWN

AND:

- `TRUE AND UNKNOWN = UNKNOWN`
- `FALSE AND UNKNOWN = FALSE`
- `UNKNOWN AND UNKNOWN = UNKNOWN`

OR:

- `TRUE OR UNKNOWN = TRUE`
- `FALSE OR UNKNOWN = UNKNOWN`
- `UNKNOWN OR UNKNOWN = UNKNOWN`

NOT:

- `NOT UNKNOWN = UNKNOWN`

Exercise 1 from Lecture - NULL with OR

```
SELECT name, salary
FROM instructor
WHERE salary > 70000 OR bonus > 50000;
```

Example row: Srinivasan with `salary = NULL, bonus = 30000`

- `salary > 70000` → UNKNOWN (NULL > 70000)
- `bonus > 50000` → FALSE (30000 > 50000)
- `UNKNOWN OR FALSE` → UNKNOWN
- Result: Row NOT included (UNKNOWN treated as FALSE)

Example row: Mozart with `salary = NULL, bonus = 55000`

- `salary > 70000` → UNKNOWN
- `bonus > 50000` → TRUE (55000 > 50000)
- `UNKNOWN OR TRUE` → TRUE
- Result: Row IS included

NULL with AND from Lecture

```
SELECT name, salary
FROM instructor
WHERE salary > 70000 AND bonus > 50000;
```

For instructor Wu with `salary = 90000` and `bonus = NULL`:

- `salary > 70000` → TRUE
- `bonus > 50000` → UNKNOWN
- `TRUE AND UNKNOWN` → UNKNOWN → treated as FALSE
- Row NOT included in result

For instructor Einstein with `salary = 95000` and `bonus = 83000`:

- `salary > 70000` → TRUE
- `bonus > 50000` → TRUE
- `TRUE AND TRUE` → TRUE
- Row IS included in result

4 Modification of the Database

4.1 Deletion

DELETE Syntax

```
DELETE FROM table_name;                  -- Delete all tuples
DELETE FROM table_name WHERE condition;  -- Delete matching tuples
```

Note: DELETE removes tuples, not the table structure.

DELETE Examples from Lecture

Delete all instructors:

```
DELETE FROM instructor;
```

Delete all Finance department instructors:

```
DELETE FROM instructor
WHERE dept_name = 'Finance';
```

4.2 Insertion

INSERT Syntax

Method 1 - Insert all values in order:

```
INSERT INTO table_name VALUES (value1, value2, ..., valueN);
```

Method 2 - Specify columns:

```
INSERT INTO table_name (col1, col2, ..., colN)
VALUES (value1, value2, ..., valueN);
```

INSERT Examples from Lecture

Insert a new course (all three are equivalent):

```
INSERT INTO course
VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

INSERT INTO course (course_id, title, dept_name, credits)
VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

INSERT INTO course
VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', null);
```

The last example inserts NULL for credits.

4.3 Updates

UPDATE Syntax

```
UPDATE table_name
SET attribute = value
WHERE condition;
```

Update with expression:

```
UPDATE table_name
SET attribute = expression
WHERE condition;
```

UPDATE Examples from Lecture

Give all instructors a 5% salary raise:

```
UPDATE instructor
SET salary = salary * 1.05;
```

Give 5% raise to instructors earning less than 70000:

```
UPDATE instructor
SET salary = salary * 1.05
WHERE salary < 70000;
```

4.4 CASE Statement in UPDATE

CASE Syntax

```
UPDATE table_name
SET attribute = CASE
    WHEN condition1 THEN value1
    WHEN condition2 THEN value2
    ...
    ELSE default_value
END
WHERE optional_condition;
```

CASE Examples from Lecture

Differential salary increase based on current salary:

```
UPDATE instructor
SET salary = CASE
    WHEN salary > 100000 THEN salary * 1.03
    ELSE salary * 1.05
END;
```

Exercise 2 from lecture - Different raises by department:

```
UPDATE instructor
SET salary = CASE
```

```
    WHEN dept_name = 'Comp. Sci.' THEN salary * 1.10
    WHEN dept_name = 'Physics' THEN salary * 1.07
    ELSE salary * 1.02
END;
```

Update department name:

```
UPDATE instructor
SET dept_name = 'Computer Science'
WHERE dept_name = 'Comp. Sci.';
```

5 Join Expressions

Join Operations Overview

Join operations take two relations and return another relation as a result.
A join operation is a Cartesian product which requires that tuples in the two relations match (under some condition).
Types of Joins:

- **Natural Join**: Matches all common attributes
- **Join ... Using**: Matches specific attributes
- **Join ... On**: Matches based on a condition
- **Outer Joins**: Preserves unmatched tuples (LEFT, RIGHT, FULL)

5.1 Natural Join

NATURAL JOIN Syntax

```
SELECT attributes
FROM table1 NATURAL JOIN table2;
```

Behavior:

- Matches tuples with same values for ALL common attributes
- Retains only ONE copy of each common column
- Common attributes are those with the same name

NATURAL JOIN Example from Lecture

List names of students along with course IDs they take:

```
SELECT name, course_id
FROM student NATURAL JOIN takes;
```

This is equivalent to:

```
SELECT name, course_id
FROM student, takes
```

```
WHERE student.ID = takes.ID;
```

Both join on the common attribute ID.

Danger of NATURAL JOIN - From Lecture

Natural join can produce unexpected results if tables share multiple common attributes.
Example Problem from Lecture:

```
SELECT name, title
FROM student NATURAL JOIN takes NATURAL JOIN course;
```

If student and course both have `dept_name`, this will join on BOTH ID and dept_name. This excludes students taking courses outside their department!
Correct approach from lecture:

```
SELECT name, title
FROM student NATURAL JOIN takes, course
WHERE takes.course_id = course.course_id;
```

This only joins student and takes on common attributes, then separately matches course_id.

5.2 Join with USING Clause

JOIN ... USING Syntax

```
SELECT attributes
FROM table1 JOIN table2 USING (attribute);
```

Behavior:

- Specifies exactly which attributes to join on
- Safer than NATURAL JOIN
- Retains only ONE copy of the join attributes

JOIN USING Example from Lecture

Safe way to join three tables:

```
SELECT name, title
FROM (student NATURAL JOIN takes) JOIN course USING (course_id);
```

This avoids the dept_name problem by explicitly specifying which attribute to use for the second join.

5.3 Join with ON Clause

JOIN ... ON Syntax

```
SELECT attributes
FROM table1 JOIN table2 ON condition;
```

Behavior:

- Most flexible join syntax
- Allows general predicates (not just equality)
- Condition written like WHERE clause but using ON keyword

JOIN ON Examples from Lecture

Basic equality join:

```
SELECT name, course_id
FROM student JOIN takes ON student.ID = takes.ID;
```

All four forms are equivalent:

```
-- Using JOIN ON
SELECT name, course_id
FROM student JOIN takes ON student.ID = takes.ID;

-- Using comma syntax
SELECT name, course_id
FROM student, takes
WHERE student.ID = takes.ID;

-- Using NATURAL JOIN
SELECT name, course_id
FROM student NATURAL JOIN takes;

-- Using JOIN USING
SELECT name, course_id
FROM student JOIN takes USING (ID);
```

5.4 Outer Joins

Outer Join Concept

An extension of the join operation that avoids loss of information.
Outer joins compute the join and then add tuples from one relation that do not match tuples in the other relation to the result of the join.
Unmatched attributes are filled with NULL.
Types:

- **LEFT OUTER JOIN**: Preserves all tuples from the left relation
- **RIGHT OUTER JOIN**: Preserves all tuples from the right relation

- **FULL OUTER JOIN**: Preserves tuples from both relations

LEFT OUTER JOIN Example from Lecture

```
SELECT *
FROM student NATURAL LEFT OUTER JOIN takes;
```

Result: All students appear in result. Students without any course enrollments have NULL for course-related attributes (course_id, semester, year, grade).
For example, if student Christian (ID=1002) has no enrollments, the result includes:

```
| 1002 | Christian | Comp. Sci. | 152 | null | null | null |
```

RIGHT OUTER JOIN Example from Lecture

```
SELECT *
FROM student NATURAL RIGHT OUTER JOIN takes;
```

Result: All course enrollments appear. If a takes record has an ID with no matching student (e.g., ID=1004), student info would be NULL:

```
| 1004 | null | null | null | CS-315 | 2023 | 10 |
```

FULL OUTER JOIN Example from Lecture

```
SELECT *
FROM student NATURAL FULL OUTER JOIN takes;
```

Result: All students AND all course enrollments appear, with NULLs where there's no match. Combines results of LEFT and RIGHT outer join.

6 Nested Subqueries

Subquery Concept

SQL provides a mechanism for nesting subqueries. A subquery is a select-from-where expression that is nested within another query.
Nesting can be done in:

- **FROM clause:** Replace table with subquery
- **WHERE clause:** B <operation> (subquery)
- **SELECT clause:** Replace attribute with subquery that generates single value

6.1 Subqueries in FROM, WHERE, SELECT

Subquery Locations from Lecture

Subquery in FROM clause:

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary) AS avg_salary
      FROM instructor
      GROUP BY dept_name)
WHERE avg_salary > 42000;
```

Subquery in WHERE clause:

```
SELECT name
FROM instructor
WHERE salary > (SELECT AVG(salary) FROM instructor);
```

Subquery in SELECT clause:

```
SELECT name, (SELECT AVG(salary) FROM instructor) AS avg_salary
FROM instructor;
```

6.2 Set Membership (IN and NOT IN)

IN Operator

```
WHERE attribute IN (subquery)
WHERE attribute NOT IN (subquery)
```

Tests whether a value is in the result set of a subquery.

IN Examples from Lecture

Find instructors whose name is neither "Mozart" nor "Einstein":

```
SELECT DISTINCT name
FROM instructor
WHERE name NOT IN ('Mozart', 'Einstein');
```

Find courses offered in both Fall 2017 and Spring 2018:

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
  AND course_id IN (SELECT course_id
                    FROM section
                    WHERE semester = 'Spring' AND year = 2018);
```

Find courses offered in Fall 2017 but NOT in Spring 2018:

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
  AND course_id NOT IN (SELECT course_id
```

```
                        FROM section
                        WHERE semester = 'Spring' AND year = 2018);
```

6.3 Set Comparison (SOME and ALL)

SOME and ALL Operators

```
WHERE attribute < SOME (subquery)   -- < at least one value
WHERE attribute > ALL (subquery)    -- > all values
```

SOME (or ANY): Condition is true if it holds for at least one value.
ALL: Condition is true if it holds for all values.
Comparison operators: $<, \leq, >, \geq, =, \neq$

Definition of SOME from Lecture

F <comp> some r $\Leftrightarrow \exists$ t $\in$ r such that (F <comp> t)
Examples:

- `(5 < some {0, 5, 6})` = true (since $5 < 6$)
- `(5 < some {0, 5})` = false
- `(5 = some {0, 5})` = true (since $5 = 5$)
- `(5 ≠ some {0, 5})` = true (since $5 \neq 0$)

SOME Example from Lecture

Find names of instructors with salary greater than at least one instructor in Comp. Sci.:

```
SELECT name
FROM instructor
WHERE salary > SOME (SELECT salary
                     FROM instructor
                     WHERE dept_name = 'Comp. Sci.');
```

Definition of ALL from Lecture

F <comp> all r $\Leftrightarrow \forall$ t $\in$ r (F <comp> t)
Examples:

- `(5 < all {0, 5, 6})` = false
- `(5 < all {6, 10})` = true
- `(5 = all {4, 5})` = false
- `(5 ≠ all {4, 6})` = true (since $5 \neq 4$ and $5 \neq 6$)

ALL Example from Lecture

Find names of instructors whose salary is greater than ALL instructors in Comp. Sci.:

```
SELECT name
FROM instructor
WHERE salary > ALL (SELECT salary
                    FROM instructor
                    WHERE dept_name = 'Comp. Sci.');
```

6.4 Test for Empty Relations (EXISTS)

EXISTS Operator

```
WHERE EXISTS (subquery)
WHERE NOT EXISTS (subquery)
```

The EXISTS construct returns TRUE if the argument subquery is nonempty.

- `EXISTS r` $\Leftrightarrow r \neq \emptyset$
- `NOT EXISTS r` $\Leftrightarrow r = \emptyset$

Often used with **correlated subqueries** where the subquery references attributes from the outer query.

EXISTS Example from Lecture

Find names of instructors who teach at least one course:

```
SELECT i.name
FROM instructor i
WHERE EXISTS (SELECT *
              FROM teaches t
              WHERE t.ID = i.ID);
```

Note: This is a correlated subquery because the inner query references `i.ID` from the outer query.
Correlation name: Variable `i` in the outer query
Correlated subquery: The inner query `SELECT * FROM teaches t WHERE t.ID = i.ID`

NOT EXISTS Example from Lecture

Find names of instructors who do not teach any course:

```
SELECT i.name
FROM instructor i
WHERE NOT EXISTS (SELECT *
                  FROM teaches t
                  WHERE t.ID = i.ID);
```

IN vs EXISTS from Lecture

Two equivalent ways to express the same query:
Using IN (uncorrelated):

```
SELECT i.name
FROM instructor i
WHERE i.ID IN (SELECT t.ID FROM teaches t);
```

Is the "left tuple" in the "right set"?
Using EXISTS (correlated):

```
SELECT i.name
FROM instructor i
WHERE EXISTS (SELECT * FROM teaches t WHERE t.ID = i.ID);
```

Is the "right set" nonempty?

6.5 Test for Duplicate Tuples (UNIQUE)

UNIQUE Operator

```
WHERE UNIQUE (subquery)
WHERE NOT UNIQUE (subquery)
```

The UNIQUE construct tests whether a subquery has any duplicate tuples in its result. UNIQUE evaluates to TRUE if a given subquery contains no duplicates.

UNIQUE Example from Lecture

Find names of instructors who do not teach duplicate courses:

```
SELECT name
FROM instructor i
WHERE UNIQUE (SELECT course_id
              FROM teaches t
              WHERE t.ID = i.ID);
```

Important: The UNIQUE predicate evaluates to TRUE on the empty set (if an instructor teaches no courses, they don't teach duplicate courses).

NOT UNIQUE Example from Lecture

Find names of instructors who teach duplicate courses:

```
SELECT name
FROM instructor i
WHERE NOT UNIQUE (SELECT course_id
                  FROM teaches t
                  WHERE t.ID = i.ID);
```

For example, instructor Brandt who teaches CS-190 in both Spring 2017 and Spring 2018 would be included.

6.6 WITH Clause

WITH Clause

The WITH clause provides a way of defining a temporary relation whose definition is available only to the query in which the WITH clause occurs.

```
WITH temp_name(col1, col2, ...) AS (subquery)
SELECT ... FROM temp_name ...;
```

WITH Clause Example from Lecture

Find all departments with the maximum budget:

```
WITH max_budget(value) AS
    (SELECT MAX(budget) FROM department)
SELECT department.name
FROM department, max_budget
WHERE department.budget = max_budget.value;
```

The temporary relation `max_budget` is available only within this query.

7 Views, Transactions and Integrity Constraints

7.1 Views

View Concept

In some cases, it is not desirable for all users to see the entire logical model (all actual relations stored in the database).
A **view** provides a mechanism to hide certain data from the view of certain users.

CREATE VIEW Syntax

```
CREATE VIEW v AS <query expression>;
```

where <query expression> is any legal SQL expression. The view name is represented by v.
Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.

View Examples from Lecture

A view of instructors without their salary:

```
CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;
```

Use the view:

```
SELECT name
FROM faculty
WHERE dept_name = 'Biology';
```

Create a view of department salary totals:

```
CREATE VIEW departments_avg_salary(dept_name, avg_salary) AS
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

7.2 Transactions

Transaction Concept

A transaction consists of a sequence of query and/or update statements and is a "unit" of work.
Transaction Control:

```
BEGIN;       -- Begin a transaction
COMMIT;      -- End and make changes permanent
ROLLBACK;    -- Reset and discard all changes
```

7.3 Integrity Constraints

Integrity Constraints Overview

Integrity constraints guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
Examples:

- A salary of a bank employee must be at least 200 an hour
- A customer must have a (non-null) phone number

Constraints on a Single Relation

- **NOT NULL**
- **PRIMARY KEY**
- **UNIQUE**
- **CHECK (P)**, where P is a predicate

CHECK Clause Example from Lecture

Ensure semester is one of fall, winter, spring or summer:

```
CREATE TABLE section (
    course_id VARCHAR(8),
    sec_id VARCHAR(8),
    semester VARCHAR(6),
    year NUMERIC(4,0),
    building VARCHAR(15),
    room_number VARCHAR(7),
    time_slot_id VARCHAR(4),
```

```
    PRIMARY KEY (course_id, sec_id, semester, year),
    CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer'))
);
```

7.4 Referential Integrity

FOREIGN KEY

Foreign keys can be specified as part of the SQL CREATE TABLE statement:

```
FOREIGN KEY (dept_name) REFERENCES department
```

By default, a foreign key references the primary-key attributes of the referenced table. SQL allows a list of attributes to be specified explicitly:

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
```

Referential Integrity Example from Lecture

```
CREATE TABLE instructor (
    ID        CHAR(5),
    name      VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    salary    NUMERIC(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
```

Valid insert:

```
INSERT INTO instructor VALUES (22222, 'Einstein', null, 95000);
```

Foreign keys can be null.
Invalid insert:

```
INSERT INTO instructor VALUES (22222, 'Einstein', 'Physics', 95000);
```

This fails if 'Physics' department doesn't exist. Foreign keys must always reference existing tuples or be null.

7.5 Handling Updates - Referential Integrity

Possible Responses to Referential Integrity Violations

When referenced data is updated or deleted:

- **Rejection** (default behavior) - Reject the operation
- **CASCADE** - Propagate updates/deletes
- **SET NULL** - Set references to NULL

Exercise 3 from Lecture - Table Creation

Create a new_student table with:

- Five attributes: ID char(5), name varchar(20), gender char(1), dept_name varchar(20), credit numeric(5, 2)
- Primary key: ID
- dept_name references department(dept_name)
- name should not be empty
- Default value of credit is 0.00
- credit should not be less than 0.00
- gender should only contain "F" and "M"

Solution:

```
CREATE TABLE new_student (
    ID        CHAR(5),
    name      VARCHAR(20) NOT NULL,
    gender    CHAR(1),
    dept_name VARCHAR(20),
    credit    NUMERIC(5,2) DEFAULT 0.00,
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name),
    CHECK (credit >= 0.00),
    CHECK (gender IN ('F', 'M'))
);
```

7.6 Cascading Actions

ON DELETE and ON UPDATE Actions

Option 1: Reject (default):

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
```

Database remains unchanged if operation would violate constraint.

Option 2: CASCADE:

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
    ON UPDATE CASCADE
    ON DELETE CASCADE
```

Updates/deletes propagate to referencing tuples.

Option 3: SET NULL:

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
    ON UPDATE SET NULL
    ON DELETE SET NULL
```

Set referencing attributes to NULL.

CASCADE Example from Lecture

```
CREATE TABLE instructor (
    ID        CHAR(5),
    name      VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    salary    NUMERIC(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Effect of UPDATE:

```
UPDATE department
SET dept_name = 'Computer Science'
WHERE dept_name = 'Comp. Sci.';
```

All instructors with dept_name = 'Comp. Sci.' are automatically updated to 'Computer Science'.

Effect of DELETE:

```
DELETE FROM department
WHERE dept_name = 'Comp. Sci.';
```

All instructors in 'Comp. Sci.' department are automatically deleted.

SET NULL Example from Lecture

```
CREATE TABLE instructor (
    ID        CHAR(5),
    name      VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    salary    NUMERIC(8,2),
    PRIMARY KEY (ID),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
        ON UPDATE SET NULL
        ON DELETE SET NULL
);
```

Effect of UPDATE:

```
UPDATE department
SET dept_name = 'Computer Science'
WHERE dept_name = 'Comp. Sci.';
```

dept_name in department updated to 'Computer Science', dept_name in instructor set to NULL.

Effect of DELETE:

```
DELETE FROM department
WHERE dept_name = 'Comp. Sci.';
```

Tuple in department deleted, dept_name in instructor set to NULL.

8 Summary

Key Takeaways

NULL Handling:

- Use IS NULL and IS NOT NULL (never = NULL or <> NULL)
- Understand three-valued logic: TRUE, FALSE, UNKNOWN
- UNKNOWN is treated as FALSE in WHERE clause
- Boolean operations: TRUE AND UNKNOWN = UNKNOWN, TRUE OR UNKNOWN = TRUE

Database Modification:

- DELETE removes tuples based on WHERE condition
- INSERT adds new tuples (values or from query)
- UPDATE modifies existing tuples
- CASE statement for conditional updates

Joins:

- NATURAL JOIN on all common attributes (dangerous if multiple common attributes)
- JOIN USING on specific attributes (safer)
- JOIN ON for general conditions (most flexible)
- Outer joins preserve unmatched tuples: LEFT, RIGHT, FULL

Nested Subqueries:

- IN/NOT IN for set membership
- SOME/ALL for set comparison
- EXISTS/NOT EXISTS for emptiness testing (correlated subqueries)
- UNIQUE/NOT UNIQUE for duplicate testing
- Subqueries can appear in FROM, WHERE, SELECT clauses
- WITH clause for temporary named relations

Views, Transactions, Integrity Constraints:

- Views provide data abstraction and hide information
- Transactions: BEGIN, COMMIT, ROLLBACK
- Constraints: NOT NULL, PRIMARY KEY, UNIQUE, CHECK, FOREIGN KEY
- Referential integrity actions: CASCADE, SET NULL, reject (default)